Assignment 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?
   BFS expands nodes in order of number of edges from the start, so the first time it reaches the goal it has used the fewest hops. It only uses edges as connections and never adds up weights. It therefore finds the fewest-hop path, which is not necessarily the cheapest one.
2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?
   You can change the frontier's pop line and the order in which neighbours are pushed. BFS uses a FIFO queue [popleft()] while DFS uses a LIFO stack [pop()]. The neighbour push order is what makes DFS order-dependent: the last neighbour pushed is expanded first, so reordering the adjacency list sends DFS down a different branch and can produce a different path. BFS is far less sensitive to this as it expands whole levels.
3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?
   The first tie you generate the goal may be visa an expensive edge. A cheaper path to the goal might be sitting in the heap or not yet discovered. Testing at generation would return that first, possibly suboptimal path. Testing at pop time guarantees that when the goal comes off the heap, it has the lowest g amount all frontier states, therefore, it is optimal.
4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?
   expanded would be more optimal as it depends only on the algorithm, graph, and tie-breaking while runtime_ms depends on CPU, load, Python version, and caching, so it isn't compatible across computers.
5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?
   GBFS ignores g, the cost already paid, and ranks states by h alone. A* uses f = g + h. This explains GBFS returned a path that was cheaper-looking but costlier overall, or wandered toward a state that looked close

to the goal but was reached expensively. Check your results to see which of those your GBFS runs showed.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?
   UCS orders by g alone so it expands Y first (3<8). GBFS orders by h alone so it expands X first (2<6). A* orders by f =g+h. X has f=10 and Y has f=9 so it expands Y first.
7. Why must *h* be expressed in the same units as the edge costs?
   A* adds h to g, and the comparison only makes sense if both are in the same units. If edges are in km and h is in miles or hours, f mixes units. h could then overestimate the true remaining cost and break admissibility, losing optimality, or it could be scaled so small that it has little effect.
8. Lab2 computes *h⋆* by running Dijkstra on the reversed graph from the goal. Why reversed?
   h*(n) is the true cheapest cost from n to the goal. Dijkstra from a single source gives distances from that source to every node. Running it from the goal on the reversed graph, with every edge flipped, gives for each node n the cost of the cheapest path from n to the goal in the original graph. This needs one run instead of one per node, which matters especially when the graph is directed or the costs are asymmetric.
9. Weighted A⋆ with *w = 1.5* changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?
   Weighted A* with w = 1.5 inflates h, which biases the search toward states that look close to the goal. That only hurts when the best-looking route differs from the cheapest one. SIN→EZE is that the cheapest route is also the one h favours, or that there are few competitive alternatives. Then inflating h doesn't change which route wins, so the cost stays the same. KUL→KEF presumably has a tempting but costlier alternative that the extra weight pulled the search toward.

Create word file and answer all the above questions. Put this in your Git Hub repo and share with me your GitHub link for AI folder as well as this file link.

Sharing GitHub Repository link

Once you have pushed your Word file to GitHub, you can easily share the repository link with me.

Here is exactly how to find and share the correct link:

1. Copy the Repository URL

1. Go to GitHub and navigate to your repository page.
2. Look at your browser's address bar. The URL should look exactly like this:
   https://github.com
3. **Copy** that link.

2. Verify Your Repository Visibility (Crucial Step)

Before sending the link, make sure your TA can actually open it:

- **If it is a Public Repository:** Your TA will be able to see it immediately.
- **If it is a Private Repository:** Your TA will get a **404 Error (Page Not Found)** unless you explicitly invite them. So better to have public one.